Topic: 1 What is Python -- The basic elements of Python

- Python is a general-purpose, dynamic, high-level, and interpreted programming language.
- It supports Object Oriented programming approach to develop applications.
- It is simple and easy to learn and provides lots of high-level data structures.
- Python is an easy-to-learn yet powerful and versatile scripting language, which makes it attractive for Application Development.
- It was created by Guido van Rossum in 1991 and further developed by the Python Software Foundation.
- There are two major Python versions: Python 2 and Python 3. Both are quite different.

Python is used for:

- Web Development (server-side),
- Software Development,
- Mathematics,
- System Scripting.

Python support following elements to perform perfect programming:

1. Data Type:

- Python support integer and float data type to hold numbers.
- Python interpreter can produce the result of numeric values.

2. Variable:

- Unlike other programming language, python have no command for declaring variable.
- You can create variable by assigning value directly to it.

EX: - x = 50

Name = “hello”

3. String:

- String is a collection of different characters.
- You can write string in single quotes(‘ ‘) as well as in double quotes (“ “).

EX: - a= ‘hello’

b=” “

4. Tuples:

- Tuples is collection of different elements and values supported by python data types.
- Tuples are enclosed in round brackets ().

EX: - a = (‘abc’, ‘xyz’, 18)

5. List:

- List is collection of element or values supported by python data types.
- List are enclosed in square brackets [ ].

EX: - a = [1,2,3,4,5]

6. Dictionary:

- A Dictionary is a collection which is unordered, changeable and indexed.
- Dictionary can be enclosed in curly brackets { } with key & value pair.

EX: - dict = {name = “Rohit”, surname = “Sharma”}

7. Operators:

- An operators are used to perform operations on variable or value.
- Python support following operators :
  - ❑ Arithmatic
  - ❑ Assignment
  - ❑ Comparison
  - ❑ Logical
  - ❑ Membership
  - ❑ Bitwise
  - ❑ Identity

Topic: 2 Branching in Python

One of the most powerful features of programming languages is branching: the ability to make decisions and execute a different set of statements based on whether one or more conditions are true.

Python programming language provided decision making statements or branching statements as follows.

1. The if statement

In Python, branching is implemented using the if statement, which is written as follows:

Syntax:

```
if condition:
    statement
```

Example:

```
i = 10
if (i > 15):
    print("10 is less than 15")
print("I am Not in if")
```

2. The else statement

If we want to print a different message if the if condition not true. This can be done by adding the else statement. It is written as follows:

Syntax:

```
if condition:
    statement
else:
    statement
```

Example:

```
i = 20
if (i < 15):
    print("i is smaller than 15")
```

```
    print("i'm in if Block")
else:
    print("i is greater than 15")
    print("i'm in else Block")
```

3. The elif statement

Python also provides an elif statement (short for "else if") to chain a series of conditional blocks.

The conditions are evaluated one by one. For the condition that evaluates to True, the block of statements below it is executed.

The remaining conditions and statements are not evaluated.

Syntax:

```
if (condition):
    statement
elif (condition):
    statement
.
.
else:
    statement
```

Example:

```
i = 20
if (i == 10):
    print("i is 10")
elif (i == 15):
    print("i is 15")
elif (i == 20):
    print("i is 20")
else:
    print("i is not present")
```

Task 1: Check if User is Eligible to Vote

```
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

Task 2: Grade System Based on Marks

```
marks = int(input("Enter your marks: "))

if marks >= 90:
    print("Grade: A+")
elif marks >= 80:
    print("Grade: A")
elif marks >= 70:
    print("Grade: B")
elif marks >= 60:
    print("Grade: C")
else:
    print("Grade: F (Fail)")
```

Task 3: Login System (Simple Username/Password Check)

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "1234":
    print("Login successful!")
else:
    print("Invalid credentials.")
```

Task 4: Check Even or Odd Number

```
num = int(input("Enter a number: "))

if num % 2 == 0:
    print(f"{num} is Even.")
else:
    print(f"{num} is Odd.")
```

Task 5: Find the Largest of Three Numbers

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

if a >= b and a >= c:
    print(f"Largest is: {a}")
```

```
elif b >= a and b >= c:
    print(f"Largest is: {b}")
else:
    print(f"Largest is: {c}")
```

Topic: 3 String in Python

- String is a collection of alphabets, words or other characters.
- Python has a built-in string class named str.
- We can create or declare a string by enclosing " " (Double Quotes) as well as ' ' Single Quotes for the statements.
- Let's see , how to declare a string value in python:

Example: - var1 = "hello"

Common String Methods

Method	**Description**
str.upper()	Converts to uppercase
str.lower()	Converts to lowercase
str.strip()	Removes whitespace from ends
str.split()	Splits string into a list of words
str.replace()	Replaces parts of the string
len(str)	Returns the length of the string

How to Access Characters from particular String: -

- We can access different characters from given string using indexing.
- In python string index will start from 0(zero).
- We cannot delete or remove the characters from the string but deleting entire string is possible using **del** keyword.

Example:

var1 = "hello world"

var2 = "python programming"

print("var1[0]:",var1[0]) output : h

print("var2[1:5]",var2[1:5]) output : ytho

How to Concatenate (join) two strings: -

To join or concatenate two strings in python, (+) plus operator can be used.

Example:

var1 = "hello "

var2 = "python programming”

print(var1 + var2)

output: hello python programming

How to update existing string: -

- You can "update" an existing string by reassigning a variable to another string.
- The new value can be related to its previous value or to a completely different string altogether.

Example:

```
var1 = 'hello world'
print ('updated string:-' , var1[:6] + 'python’)
```
output: hello python

Multiline Strings

You can assign a multiline string to a variable by using three quotes:

Example:

```
a = """Hello,
Welcome to our College,
Rajkot."""
print(a)
```

Deleting Entire String

Deletion of the entire string is possible with the use of del keyword.

Example:

```
String1 = "Hello, I'm a Programmer"
print("Initial String: ")
print(String1)
del String1
print("\nDeleting entire String: ")
print(String1)
```

Topic: 4 Input in Python

- To input value python, provide one of the built – in function named input().
- You can ask user to input particular value from userside.
- You can call input() and wait for user to enter the data.
- In python2 you can use row_input() to get value from userside, while in python3 you have input() to get value from user.

Example:

```
num = input ("Enter number :")
print(num)
name1 = input("Enter name : ")
print(name1)
print ("type of number", type(num))
print ("type of name", type(name1))
```

By default, the input() function takes input as a string so if we need to enter the integer or float type input then the input() function must be type casted.

Example:

```
name = input("Enter your name: ")
```

```
age = int(input("Enter your age: "))
marks = float(input("Enter your marks: "))
print("The name is:", name)
print("The age is:", age)
print("The marks is:", marks)
```

We can see in the above code where we type casted the user input into int and float.

Topic: 5 Iteration in Python

An iterator in Python is an object that is used to iterate over iterable objects like lists, tuples, dicts, and sets. The Python iterators object is initialized using the iter() method. It uses the next() method for iteration.

__iter__(): The iter() method is called for the initialization of an iterator. This returns an iterator object

__next__(): The next method returns the next value for the iterable.

Example:

```
string = "HELLO"
ch_iterator = iter(string)

print(next(ch_iterator))
print(next(ch_iterator))
print(next(ch_iterator))
print(next(ch_iterator))
print(next(ch_iterator))
```

Example:

Return an iterator from a tuple, and print each value:

```
mytuple = ("apple", "banana", "cherry")
myit = iter(mytuple)
print(next(myit))
print(next(myit))
print(next(myit))
```

Working of for loop for Iterators

The for loop in Python is used to iterate over a sequence of elements, such as a list, tuple, or string.

Example:

```
my_list = [1, 2, 3, 4, 5]
iterator = iter(my_list)
for element in iterator:
    print(element)
```

There is a much easier way to do this also by using a For loop. You don't need to explicitly call the iter() and next() functions.

Example:

```
mytuple = ("apple", "banana", "cherry")
for x in mytuple:
  print(x)
```

Using range() Method

Using the range() function we will be iterating over lists in Python.

Syntax: range(start, stop, step)

The range() function takes 3 arguments.

Example:

```
items = [3, 1, 4, 3.14]
length = len(items)
for i in range(length):
   print[items[i])
```

Nested Loops

Python programming language allows to use one loop inside another loop.

Example:

```
for i in range(1, 5):
   for j in range(i):
      print(i, end=' ')
   print()
```

Output:

1

2 2

3 3 3

4 4 4 4

Using while Loop

We will be iterating over lists in Python using the while loop available in Python.

Syntax:

```
while(condition):
        ... code statements ...
        increment iterator
```

The while loop only allows the code statements to be executed if and only if, the condition provided to the while loop returns true.

Example:

```
items = [3, 1, 4, 3.14]
length = len(items)
i = 0
while i < length:
        print(items[i])
        i += 1
```

Topic: 6 Function in Python

Python provides the following types of functions −

Built-in functions

Python's standard library includes number of built-in functions. Some of Python's built-in functions are print(), int(), len(), sum(), etc.

User-defined functions

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.

In Python a function is defined using the def keyword.

Example:

```
def my_function():
  print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

Example:

```
def my_function():
```

```
  print("Hello from a function")
my_function()
```

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

Example:

```
def my_function(fname):
  print("Welcome " + fname)
my_function("Rahul")
my_function("Rohit")
my_function("Shubhman")
```

Task: 1 Count vowels and consonants in a string

```
def count_vowels_consonants(s):
    vowels = "aeiouAEIOU"
    v = c = 0
    for char in s:
        if char.isalpha():
            if char in vowels:
                v += 1
            else:
                c += 1
    return v, c
```

```
v, c = count_vowels_consonants("Python Programming")
print(f"Vowels: {v}, Consonants: {c}")
```

Task:2 Remove duplicate characters

```
def remove_duplicates(s):
    result = ""
    for char in s:
        if char not in result:
            result += char
    return result

print(remove_duplicates("programming"))
```

Task: 3 Count frequency of each character

```
from collections import Counter

s = "hello world"
freq = Counter(s)
print(freq)
```

Task: 4 Capitalize first letter of each word

```
s = "hello world from python"
capitalized = s.title()
print(capitalized)
```

Topic: 7 Scope in python

Local Scope

A variable created inside a function belongs to the local scope of that function, and can only be used inside that function.

Example:

```
def myfunc():
  x = 300
  print(x)
myfunc()
```

Global Scope

A variable created in the main body of the Python code is a global variable and belongs to the global scope.

Global variables are available from within any scope, global and local.

Example:

```
x = 300
def myfunc():
  print(x)
myfunc()
print(x)
```

Naming Variables

If you operate with the same variable name inside and outside of a function, Python will treat them as two separate variables.

Example:

```
x = 300
def myfunc():
  x = 200
  print(x)
myfunc()
print(x)
```

Topic: 8 Global Keyword

If you need to create a global variable, you can use the global keyword.

The global keyword makes the variable global.

Example:

```
def myfunc():
  global x
  x = 300
myfunc()
print(x)
```

Example : To change the value of a global variable inside a function.

```
x = 300
def myfunc():
  global x
  x = 200
myfunc()
print(x)
```

Topic: 9 Recursive Function

A function can call other functions. It is even possible for the function to call itself. These types of construct are termed as recursive functions.

Syntax:

```
def func(): <--
            |
            | (recursive call)
            |
    func() ---
```

Example:

```
def Recur_facto(n):
    if (n == 0):
        return 1
    return n * Recur_facto(n-1)
print(Recur_facto(6))
```

Topic: 10 Module in Python

Consider a module to be the same as a code library.

A file containing a set of functions you want to include in your application.

Create a Module

To create a module just save the code you want in a file with the file extension .py:

Example:

Save this code in a file named mymodule.py

```
def greeting(name):
  print("Hello, " + name)
```

Use a Module

Now we can use the module we just created, by using the import statement:

Example:

Import the module named mymodule, and call the greeting function:

```
import mymodule

mymodule.greeting("Ravi")
```

Variables in Module

The module can contain functions, as already described, but also variables of all types (arrays, dictionaries, objects etc):

Example:

Save this code in the file mymodule.py

```
person1 = {
  "name": “Ravi",
  "age": 36,
  "country": “India"
}
```

Example:

Import the module named mymodule, and access the person1 dictionary:

```
import mymodule

a = mymodule.person1["age"]

print(a)
```

Re-naming a Module

You can create an alias when you import a module, by using the as keyword:

Example:

Create an alias for mymodule called mx:

```
import mymodule as mx
a = mx.person1["age"]
print(a)
```

Import From Module

You can choose to import only parts from a module, by using the from keyword.

Example:

The module named mymodule has one function and one dictionary:

```
def greeting(name):
  print("Hello, " + name)
person1 = {
  "name": “Ravi",
  "age": 36,
  "country": “India"
}
```

Example: Import only the person1 dictionary from the module:

```
from mymodule import person1
print (person1["age"])
```

Topic: 11 Python File Handling

- Python supports file handling and allows users to handle files, to read and write files, along with many other file handling options, to operate on files.
- File handling in Python allows you to perform a wide range of operations, such as creating, reading, writing, appending, renaming, and deleting files.
- File handling in Python is highly flexible, as it allows you to work with different file types, e.g. text files, binary files, CSV files, etc.

Working of open() Function in Python

Before performing any operation on the file like reading or writing, first, we have to open that file.

For this, we should use Python’s inbuilt function open() but at the time of opening, we have to specify the mode, which represents the purpose of the opening file.

f = open(filename, mode)

File Modes:

r: open an existing file for a read operation.

w: open an existing file for a write operation. If the file already contains some data then it will be overridden but if the file is not present then it creates the file as well.

a: open an existing file for append operation. It won't override existing data.

r+: To read and write data into the file. The previous data in the file will be overridden.

w+: To write and read data. It will override existing data.

a+: To append and read data from the file. It won't override existing data.

Working in Read mode

There is more than one way to read a file in Python.

Example 1: The open command will open the file in the read mode and the for loop will print each line present in the file.

```
file = open('demo.txt', 'r')
for content in file:
    print (content)
```

Example 2: In this example, we will extract a string that contains all characters in the file then we can use file.read().

```
file = open("demo.txt", "r")
print (file.read())
```

Example 3: In this example, we will see how we can read a file using the with statement.

```
with open("demo.txt") as file:
    data = file.read()
print(data)
```

Example 4: Another way to read a file is to call a certain number of characters like in the following code the interpreter will read the first five characters of stored data and return it as a string:

```
file = open(“demo.txt", "r")
print (file.read(5))
```

Creating a File using the write() Function

Just like reading a file in Python, there are a number of ways to write in a file in Python.

Working in Write Mode

Example 1: In this example, we will see how the write mode and the write() function is used to write in a file. The close() command terminates all the resources in use and frees the system of this particular program.

```
file = open('demo.txt','w')
file.write("This is the write command")
file.write("It allows us to write in a particular file")
file.close()
```

Example 2: We can also use the written statement along with the with() function.

```
with open("file.txt", "w") as f:
   f.write("Hello World!!!")
```

Working of Append Mode

Example:

```
file = open(‘demo.txt', 'a')
file.write("This will add this line")
file.close()
```

Implementing all the functions in File Handling

In this example, we will cover all the concepts of file handling. We will also see how we can delete a file using the remove() function from Python os module.

```
import os

def create_file(filename):
    try:
        with open(filename, 'w') as f:
            f.write('Hello, world!\n')
        print("File " + filename + " created successfully.")
    except IOError:
        print("Error: could not create file " + filename)


def read_file(filename):
    try:
        with open(filename, 'r') as f:
            contents = f.read()
            print(contents)
    except IOError:
        print("Error: could not read file " + filename)


def append_file(filename, text):
    try:
        with open(filename, 'a') as f:
            f.write(text)
        print("Text appended to file " + filename + " successfully.")
    except IOError:
        print("Error: could not append to file " + filename)


def rename_file(filename, new_filename):
```

```
    try:
        os.rename(filename, new_filename)
        print("File " + filename + " renamed to " + new_filename + " successfully.")
    except IOError:
        print("Error: could not rename file " + filename)

def delete_file(filename):
    try:
        os.remove(filename)
        print("File " + filename + " deleted successfully.")
    except IOError:
        print("Error: could not delete file " + filename)

filename = "example.txt"
new_filename = "new_example.txt"

create_file(filename)
read_file(filename)
append_file(filename, "This is some additional text.\n")
read_file(filename)
rename_file(filename, new_filename)
read_file(new_filename)
delete_file(new_filename)
```

Topic: 12 Python Tuples

Python Tuple is a collection of objects separated by commas.

In some ways, a tuple is similar to a Python list in terms of indexing, nested objects, and repetition but the main difference between both is Python tuple is immutable, unlike the Python list which is mutable.

Some Characteristics of Tuples in Python

- We can find items in a tuple since finding any item does not make changes in the tuple.
- One cannot add items to a tuple once it is created.
- Tuples cannot be appended or extended.
- We cannot remove items from a tuple once it is created.

Creating Python Tuples

There are various ways by which you can create a tuple in Python. They are as follows:

- **Using round brackets**
- **With one item**
- **Tuple Constructor**

Create Tuples using Round Brackets ()

To create a tuple, we will use () operators.

Example:

```
var = ("BCA", "BSCIT", "MSCIT", "MCA")

print(var)
```

Create a Tuple With One Item

Python 3.11 provides us with another way to create a Tuple.

Example:

```
values : tuple[int | str,] = (1,2,4,“BCA")

print(values)
```

In case you are generating a tuple with a single element, make sure to add a comma after the element. Let us see an example of the same.

Example:

```
mytuple = (“BCA",)
print(type(mytuple))

#NOT a tuple
mytuple = (“BCA")
print(type(mytuple))
```

Tuple Constructor in Python

To create a tuple with a Tuple constructor, we will pass the elements as its parameters.

Example:

```
tuple_constructor = tuple(("PHP", "PYTHON", "JAVA"))
print(tuple_constructor)
```

Example of immutable tuple:

```
mytuple = (1, 2, 3, 4, 5)

# tuples are indexed
print(mytuple[1])
print(mytuple[4])

# tuples contain duplicate elements
mytuple = (1, 2, 3, 4, 2, 3)
print(mytuple)
```

```
# adding an element
mytuple[1] = 100
print(mytuple)
```

Accessing Values in Python Tuples

Tuples in Python provide two ways by which we can access the elements of a tuple.

- **Using a positive index**
- **Using a negative index**

Python Access Tuple using a Positive Index

Using square brackets, we can get the values from tuples in Python.

Example:

```
var = ("BCA", "BSCIT", "MSCIT")

print("Value in Var[0] = ", var[0])
print("Value in Var[1] = ", var[1])
print("Value in Var[2] = ", var[2])
```

Access Tuple using Negative Index

In the above methods, we use the positive index to access the value in Python, and here we will use the negative index within [].

Example:

```
var = (1, 2, 3)
print("Value in Var[-1] = ", var[-1])
print("Value in Var[-2] = ", var[-2])
print("Value in Var[-3] = ", var[-3])
```

Concatenation of Python Tuples

To Concatenation of Python Tuples, we will use plus operators(+).

Example:

```
tuple1 = (0, 1, 2, 3)
tuple2 = ('python', ‘php')
print(tuple1 + tuple2)
```

Nesting of Tuples

A nested tuple in Python means a tuple inside another tuple.

Example:

```
tuple1 = (0, 1, 2, 3)
tuple2 = ('python', ‘php')
tuple3 = (tuple1, tuple2)
print(tuple3)
```

Deleting a Tuple in Python

We can delete a tuple using ‘del’ keyword. The output will be in the form of error because after deleting the tuple, it will give an error.

Example:

```
tuple3 = ( 0, 1)
del tuple3
print(tuple3)
```

Topic: 13 Lists and Mutability

Mutability refers to whether an object can be changed after it’s created.

- Mutable objects can be changed.
- Immutable objects cannot be changed.

Lists are mutable in Python.
This means that you can:

- Add items
- Remove items
- Change items in-place

Example of Mutable Behavior in Lists

```
# Original list
fruits = ["apple", "banana", "cherry"]

# Change an element
fruits[1] = "blueberry"
print(fruits)  # ['apple', 'blueberry', 'cherry']

# Add an element
fruits.append("mango")
print(fruits)  # ['apple', 'blueberry', 'cherry', 'mango']

# Remove an element
fruits.remove("apple")
print(fruits)  # ['blueberry', 'cherry', 'mango']
```

List Assignment vs Copy

```
# List assignment (reference)
a = [1, 2, 3]
b = a
b[0] = 100
print(a)  # [100, 2, 3] → changes in 'b' reflect in 'a'

# List copy (new object)
a = [1, 2, 3]
b = a.copy()
b[0] = 100
print(a)  # [1, 2, 3] → 'a' is unchanged
```

Lists support many mutable operations: append(), insert(), remove(), pop(), clear(), sort(), reverse().

Checking Mutability

You can use the id() function to check if two variables refer to the same object in memory:

```
x = [1, 2]
y = x
print(id(x), id(y))
```

Topic: 14 Functions as Objects

In Python, functions are first-class objects.
This means:
Functions can be:

- Assigned to variables
- Passed as arguments
- Returned from other functions
- Stored in data structures (like lists, dicts)

1. Assigning Functions to Variables

```
def greet():
    print("Hello!")

say_hello = greet
say_hello()
```

2. Passing Functions as Arguments

```
def function1(text):
    return text.upper()

def function2(text):
    return text.lower()

def main_function(func):
    result = func("Hello World")
    print(result)
```

```
main_function(function1)
main_function(function2)
```

3. Returning Functions from Functions

```
def outer():
    def inner():
        print("Inside inner function")
    return inner

func = outer()
func()
```

4. Storing Functions in a List or Dictionary

```
def add(x, y): return x + y
def sub(x, y): return x - y

operations = [add, sub]
print(operations[0](5, 3))
print(operations[1](5, 3))

ops_dict = {'add': add, 'sub': sub}
print(ops_dict['add'](10, 4))
```

Topic: 15 Tuples vs Lists in Python

Both tuples and lists are sequences used to store multiple items — but they behave differently in important ways.

1. Mutability

Feature	List	Tuple
Mutable	Yes	No
Can be changed	Yes	No

```
# List: mutable
my_list = [1, 2, 3]
my_list[0] = 100
```

```
print(my_list)  # [100, 2, 3]

# Tuple: immutable
my_tuple = (1, 2, 3)
# my_tuple[0] = 100  ✗ Error
```

2. Syntax

```
# List
fruits = ["apple", "banana", "cherry"]

# Tuple
coordinates = (10.0, 20.5)
```

3. Use Cases

Lists: when data can change
Tuples: when data should not change (e.g. coordinates, fixed data)

4. Performance

Tuples are faster and use less memory than lists.
Good for read-only data.

5. Tuples can store any object, including lists.

```
mixed = (1, 2, [3, 4])
mixed[2][0] = 99
print(mixed)  # (1, 2, [99, 4])
```

Even though the tuple itself is immutable, the list inside it is mutable.

Example:

```
# List of students
students = ["Anil", "Brijesh", "Sachin"]

# Tuple of subject codes
subjects = ("MATH101", "PHY102", "CS103")
# Print each student with subject
for i in range(len(students)):
    print(f"{students[i]} has {subjects[i]}")
```

Topic: 16 Dictionaries

A dictionary in Python is a collection of key-value pairs.
It's unordered (in older Python versions), mutable, and indexed by keys (not positions).

1. Basic Syntax

```
# Creating a dictionary
student = {
   "name": "Anil",
   "age": 21,
   "course": "BCA"
}
```

Keys: "name", "age", "course"
Values: "Anil", 21, "BCA"

2. Accessing and Modifying Data

```
# Accessing
print(student["name"])

# Changing a value
student["age"] = 22

# Adding a new key-value pair
student["grade"] = "A"

# Deleting a key
del student["course"]
```

3. Useful Dictionary Methods

Method	Description
dict.keys()	Returns all keys
dict.values()	Returns all values
dict.items()	Returns all (key, value) pairs
dict.get(key)	Safer way to get a value
dict.update()	Updates from another dict
dict.clear()	Removes all items

```
print(student.keys())     # dict_keys(['name', 'age', 'grade'])
print(student.get("name"))  # Alice
print(student.get("salary", "Not Found"))  # Not Found
```

4. Looping Through a Dictionary

```
for key, value in student.items():
    print(key, ":", value)
```

5. Nested Dictionaries

```
students = {
    "101": {"name": "Anil", "age": 21},
    "102": {"name": "Brijesh", "age": 22}
}
print(students["101"]["name"])
```

Example:

```
# Program to store and display employee data

employee = {}

employee["id"] = input("Enter ID: ")
employee["name"] = input("Enter name: ")
employee["salary"] = float(input("Enter salary: "))

print("\nEmployee Details:")
for k, v in employee.items():
    print(k.capitalize(), ":", v)
```